FeedbackSense — Interview Q&A

Product walkthrough answers for a React + AI dashboard demo. Keep this as a prep sheet, not a script to read word-for-word.

30-second opener
FeedbackSense is a React dashboard for managers. You upload customer comments, AI labels category, sentiment, and priority, History keeps every analyzed batch, and Trends compares years. Demo AI uses keyword rules. Live AI calls Claude on the server. I did not train a machine-learning model.

1. Product

What problem does this solve, and for whom?

A manager, for example in a food company, receives comments from many years and cannot see whether quality issues are getting better. FeedbackSense classifies each comment and lets them review the latest batch, keep old batches, and compare 2022–2026.

Walk me through a manager's flow.

1. Open Upload Feedback.
2. Paste comments or upload a CSV.
3. Click Analyze Feedback.
4. Dashboard shows KPIs, charts, and the table.
5. History stores that analysis.
6. Trends compares dated history across years.
7. Click a KPI or filter to drill into exact comments.

Why History and Trends if Dashboard already has charts?

Dashboard and Analytics show the **latest analysis only**.
History stores **every successful analysis**.
Trends builds year-over-year charts from **dated History**, not from a new upload.

Clear Results empties Dashboard. It does not delete History.

What is a KPI here, and why four cards?

KPI means Key Performance Indicator: a number a manager checks first.

Card	What it counts
Total Feedback	All analyzed rows
High Priority	`priority === High`

Negative Sentiment	`sentiment === Negative`
Feature Requests	`category === Feature Request`

They are four counts from three different fields, not four values on one scale. Clicking a card filters the table.

How do dates work? What are the three scenarios?

1. **File has dates** — Use `sample-history-feedback.csv`. Do not change the year dropdown. Each row keeps its CSV date.
2. **No dates, hurried upload** — Year for this batch defaults to the current year. The batch still appears in History and Trends using that year or the import time. The app does not invent a fake customer day.
3. **Manager wants a specific year** — Select 2022 (or another year), then Analyze. That batch is stored under that year.

If the manager chooses **No year (unknown)**, History can show **Imported 2026** from analyze time. Trends can use the import year as a fallback.

File dates always win over the dropdown.

If I upload 2022 and 2025 files, what should I see?

Analyze the 2022 file with year 2022 (or dates in the file). Then analyze the 2025 file. History contains both. Trends can compare 2022 vs 2025. Dashboard shows only the latest file until you look at History.

What would you not claim to a customer?

Do not claim the AI “knows” food quality unless the stored data supports it. Trend insights are generated only from stored counts. Sample Dashboard data is demo data, not History, until the user analyzes a file.

How is this different from Excel + ChatGPT?

The product is the workflow: upload, validate, classify, persist, filter, chart, and compare years in one UI. ChatGPT in Excel does not give a reusable manager dashboard with saved history.

---

2. AI vs machine learning

Is FeedbackSense a machine learning project?

No. It is an **AI-enabled React product**. There is no trained model, no dataset pipeline, and no TensorFlow or scikit-learn.

Demo: keyword rules in `mockAi.js`.
Live: Claude via `/api/analyze-feedback`.
The API key never sits in frontend code.

How are category, sentiment, and priority decided?

Each comment gets **three independent labels**:

Field	Meaning	Values
Category	What kind of comment	Bug, Feature Request, Praise, Complaint
Sentiment	How they feel	Positive, Neutral, Negative
Priority	How urgent	High, Medium, Low

React does not assign these. The analysis step does. React only displays, counts, and filters.

Why are those three fields separate?

A customer can be unhappy (Negative) about a crash (Bug, High). Another can ask for dark mode (Feature Request, Neutral, Medium) without being angry. Emotion, urgency, and topic are different questions.

What does mock AI actually do?

It scans the comment in order:

1. crash / does not work / broken / bug / Safari / incomplete → Bug, usually Negative + High
2. please add / can you add / would be helpful → Feature Request, Neutral + Medium
3. love / fantastic / quickly → Praise, Positive + Low
4. support / slow / three days → Complaint, Negative + High
5. anything else → Complaint, Neutral + Medium

Why not put the API key in React?

Anything in the frontend can be copied from the browser. The key stays on the server. The client posts comments to `/api/analyze-feedback`.

What if Claude returns bad JSON?

Results are validated: non-empty text, reason, and allowed category, sentiment, and priority. Invalid payloads fail with a friendly error. The user can retry.

How do you stop invented trend claims?

Historical insight sentences are built from stored counts only, for example volume from 8 in 2022 to 5 in 2026. If there is not enough dated data, no insight is shown.

If this were an ML project, what would change?

You would collect labeled comments, train a classifier, measure accuracy, and host `/predict`. The same React UI could call that API. You add ML when volume, cost, or privacy makes Claude a bad fit.

When do you use rules, an LLM, or ML?

Situation	Use
Few obvious keywords, MVP	Rules
Messy language, ship fast, little labeled data	LLM (Claude)
Huge volume, privacy, must prove accuracy	Your own ML model
Store, filter, compare years	App + database, not ML

---

3. React and architecture

Why Vite + React, not Next.js?

This is a client dashboard. Vite is a fast SPA setup. Server work is only the Claude wrapper. Next.js would help SEO or server rendering, which this product does not need yet.

Why Redux Toolkit instead of Context?

Analysis, loading, errors, latest items, and history are shared across Dashboard, History, Trends, and Settings. Redux Toolkit gives a clear slice and async thunks. Context is enough for theme or a single value, not this data flow.

What is in Redux vs local React state?

Redux: `items`, `historyItems`, `loading`, `error`, `isSampleData`.
Local state: search, sidebar section, paste text, file upload, table filters, modal, toast, batch year.

Why upload then Analyze, not auto-analyze?

Two-step is standard in tools like this. The user can check the file, set a year, then spend the API call. Accidental drops do not fire analysis.

How does a KPI click filter the table?

The card sets filter state, for example High Priority sets priority to High. The table uses `filterFeedback`. From Analytics, it also switches to the Feedback table and scrolls to it.

How do tabs work without a router?

`activeSection` in App chooses Dashboard, Upload, History, Trends, Analytics, or Settings. Dashboard stays mounted but hidden so paste and filters are not lost.

How are loading and errors handled?

Analyze is a `createAsyncThunk`. Pending shows a loading state. Rejected maps codes like `EMPTY_INPUT` or `NETWORK` to friendly messages. A toast confirms success.

---

4. Data, CSV, and persistence

What does one analyzed item look like?

`id`, `text`, `category`, `sentiment`, `priority`, `reason`, plus `date`, `year`, `source`, `analyzedAt`. Priority is the severity field. We extended this model instead of inventing a new one.

Where is history stored?

Redux `historyItems`, persisted in `localStorage` as `feedbacksense-history`. There is no backend database in this portfolio build.

Why localStorage, not a database?

Smallest persistence that keeps History after refresh, with no server. Limits: one browser, size cap, not multi-user. A real customer would use Postgres (or similar) behind an API.

Clear results vs Clear history?

Clear results — latest Dashboard/Analytics items only.
Clear history — all saved History and Trends data.

How is CSV parsed?

Papa Parse. A `feedback` column (or similar) is the comment. A `date` or `year` column becomes the record date. CSV, TSV, and TXT are accepted. Dates are parsed without using `Date()` in a way that would shift timezone days.

If someone pastes `date,feedback` CSV into the text box, the app still extracts dates instead of saving the whole line as the comment.

What file should I demo?

`public/sample-history-feedback.csv` — 31 comments dated 2022–2026. Leave the year dropdown as the default. Do not paste that file into the left box; upload it.

`sample-feedback.csv` has no dates. Use it to show batch year.

---

5. History, Trends, and charts

What does History show?

Saved analyzed records, year range, category, sentiment, priority filters, KPIs, search, and a date column. Opening a row shows labels, AI reason, source, and imported date.

What does Trends show?

Volume by year, priority by year, category by year, sentiment by year, top categories, a comparison table, and AI Historical Insight. It reads dated History only (including import-year fallback). It does not upload files.

Why Recharts?

It was already in the project, works with React, and is enough for bars, lines, and pie charts without a custom chart engine.

---

6. UI decisions

Why were KPI icons changed?

The old mint squares and plus/list glyphs looked like buttons. Icons are now decorative (comment bubble, alert, face, lightbulb). The **card** can still be clicked after analysis; the icon should not look like a separate control.

How does mobile work?

Cards replace the table. The sidebar becomes a menu. Grids collapse to one column.

Empty and error states?

Empty charts, empty History, empty Trends, invalid file type, empty CSV, failed analysis, and sample-data banner are all explicit so the demo never looks “broken” without explanation.

---

7. Tradeoffs and next steps

Biggest technical compromise?

History is browser `localStorage`, and demo classification is keyword rules. Both are honest portfolio choices, not production multi-tenant architecture.

What breaks at 100,000 comments?

localStorage size, rendering the full table, and analyzing everything in one request. You would add a database, pagination, background jobs, and batch APIs.

How would you add companies and login?

Auth, `companyId` on every record, API scoped to the user, and a real database. The UI can stay similar.

How would you test it?

Unit tests for CSV date parsing, filters, and year aggregations. Component tests for KPI click-to-filter. One E2E path: upload dated CSV → Analyze → History dates → Trends charts.

What would you build in the next two weeks?

Auth, Postgres, pagination, tests, live Claude in staging, export CSV, and optional “compare by import year” vs “feedback year” in the UI.

---

8. Live-coding they might ask

Add a Medium Priority KPI.

Add `mediumPriority: items.filter(i => i.priority === 'Medium').length` in `getSummaryMetrics`, a fifth card, and a click handler that sets priority to Medium.

Filter History by source filename.

Source is already stored. Add a dropdown of unique `item.source` values and include it in `filterHistory`.

Stop duplicate CSV uploads.

Hash text + date, skip ids already in `historyItems`, or ask “This file was analyzed. Append anyway?”

High-priority count by year.

`getPriorityByYear(items, years)` already returns High / Medium / Low per year. That is the Trends table.

---

9. Questions you can ask the interviewer

- Do you want a product walkthrough or a code walkthrough first?
- Should I show the dated CSV path or the no-date year dropdown?
- For your team, would this sit in front of Claude, an in-house model, or both?

---

10. Demo checklist (5 minutes)

1. Dashboard — sample KPIs, click High Priority.
2. Upload `sample-history-feedback.csv` — file dates, Analyze.
3. Open a row — labels + AI reason.
4. History — 2022–2026, filter Complaint.
5. Trends — volume and high-priority over years.
6. Settings — Clear results, History still there.

Close with: “Upload once, AI classifies, History keeps it, Trends shows whether the issue is getting better or worse.”